

Вопрос 1: Factory Method

Назначение

Фабричный метод определяет интерфейс для создания объектов, оставляя подклассам решение о том, какие именно классы будут создаваться. Этот паттерн позволяет делегировать создание экземпляров подклассам, что обеспечивает гибкость и улучшает расширяемость кода.

Другие названия

Иногда фабричный метод называют **Virtual Constructor** (виртуальный конструктор).

Мотивация

Этот паттерн особенно полезен в каркасах, где необходимо поддерживать и управлять отношениями между объектами, создавая их только тогда, когда это требуется. Например, в приложениях с несколькими документами каркас должен создавать объекты типа Document, но не знает заранее их конкретные классы. Фабричный метод решает эту проблему, вынося информацию о конкретных подклассах за пределы основного каркаса.

Применимость

Фабричный метод применяется, если:

- Классу неизвестно, какие конкретные классы объектов ему нужно создавать.
 - Класс делегирует создание объектов подклассам.
 - Требуется инкапсуляция логики выбора конкретных подклассов для создания объектов.
-

Структура

1. **Product**: определяет интерфейс создаваемых объектов.
2. **ConcreteProduct**: реализует интерфейс Product.
3. **Creator**: объявляет фабричный метод, который возвращает объект типа Product. Может предоставлять реализацию по умолчанию.
4. **ConcreteCreator**: переопределяет фабричный метод, чтобы возвращать объекты конкретного класса.

Пример структуры:

Creator

└─ FactoryMethod()

└─ AnOperation()

└─ ConcreteCreator

└─ FactoryMethod() → ConcreteProduct

Преимущества

1. **Инкапсуляция логики создания объектов:** клиентский код работает только с интерфейсами Product, не заботясь о деталях реализации.
2. **Гибкость:** позволяет легко добавлять новые типы продуктов, изменяя только конкретные фабрики.
3. **Масштабируемость:** подклассы могут предоставлять собственные реализации фабричного метода.

Недостатки

1. Увеличение сложности системы из-за необходимости создания подклассов для каждого типа продукта.
2. Иногда требуется создавать подклассы даже для небольших изменений в логике создания.

Пример реализации

На языке Java фабричный метод может выглядеть следующим образом:

```
interface Product {  
    void use();  
}
```

```
class ConcreteProduct implements Product {  
    @Override  
    public void use() {  
        System.out.println("Using ConcreteProduct");  
    }  
}
```

```
abstract class Creator {  
    public abstract Product createProduct();  
}
```

```
class ConcreteCreator extends Creator {  
    @Override  
    public Product createProduct() {  
        return new ConcreteProduct();  
    }  
}
```

```
}  
  
}
```

Родственные паттерны

- **Абстрактная фабрика:** предоставляет интерфейс для создания семейства связанных объектов.
- **Строитель:** акцентирует внимание на пошаговом создании сложных объектов.
- **Прототип:** может заменить фабричный метод, клонируя заранее заданные прототипы продуктов.

Если нужны дополнительные примеры или разъяснения, дайте знать!

Вопрос 2: 5 признаков сложных систем по Гради Бучу

1 Иерархичность

- "Сложные системы часто являются иерархическими и состоят из взаимозависимых подсистем, которые в свою очередь также могут быть разделены на подсистемы, и т. д., вплоть до самого низкого уровня."
- Иерархическая структура позволяет понимать и описывать систему через её части.

2 Произвольность определения элементарных компонент

- "Выбор, какие компоненты в данной системе считаются элементарными, относительно произволен и в большой степени оставляется на усмотрение исследователя."
- Уровень элементарности зависит от задачи или точки зрения исследователя.

3 Более сильная внутренняя связь

- "Внутрикомпонентная связь обычно сильнее, чем связь между компонентами. Это обстоятельство позволяет отделять «высокочастотные» взаимодействия внутри компонентов от «низкочастотной» динамики взаимодействия между компонентами."
- Такое разделение упрощает изучение частей системы.

4 Ограниченное разнообразие типов подсистем

- "Иерархические системы обычно состоят из немногих типов подсистем, по-разному скомбинированных и организованных."
- Это свойство упрощает проектирование и использование общих компонентов.

5 Эволюционное развитие

- "Любая работающая сложная система является результатом развития работавшей более простой системы... Сложная система, спроектированная «с нуля», никогда не заработает. Следует начинать с работающей простой системы."
- Системы должны эволюционировать, начиная с простых, чтобы достичь стабильности.